RAPPORT D'ANALYSE DES PERFORMANCES

Système de gestion d'un entrepôt logistique

Temps d'exécution des requêtes principales

Auteurs	Jad ABDUL RAZEK, Matthias LUTZ, Abdelmalik TELITEL
Enseignant	Alexandre Lombard
Date	Novembre 2026

Mesures réalisées le 04/11/2026 à 16:48 sur MariaDB 10.4.32-MariaDB (poste local), base alimentée par `sql/installer.php`. Chaque requête est exécutée 20 fois après une exécution de chauffe ; le temps retenu est la **médiane** (en millisecondes, récupération des lignes comprise).

Volumétrie

Table	Lignes (estimation InnoDB)
`passage_etape`	23 232
`mouvement_stock`	16 875
`ligne_prelevement`	12 005
`lot`	3 941
`rendez_vous`	3 679
`commande`	3 447
`passage`	3 431
`reception`	1 703
`stock`	746

Requêtes principales

#	Requête	Temps médian	Sans index	Plan d'exécution
1	**Planning d'une journée** — Rendez-vous non annulés du jour, tous quais (planning.php)	0.22 ms	1.52 ms (×6.8)	r : index `ix_rdv_debut` (71 l.) ; v : index `PRIMARY` (1 l.) ; t : index `PRIMARY` (1 l.)
2	**Contrôle de chevauchement** — Test exécuté par le déclencheur à chaque prise de rendez-vous	0.08 ms	1.21 ms (×15.2)	r : index `ix_rdv_quai_creneau` (223 l.)
3	**Camions sur site** — Passages dont la dernière étape n'est pas le départ (flux.php)	1.43 ms	–	v : index `immatriculation` (123 l.) ; p : index `fk_passage_vehicule` (13 l.) ; te : index `PRIMARY` (1 l.)
4	**État des quais en temps réel** — Vue v_quai_temps_reel (statut calculé, camion à quai, prochain RDV)	2.69 ms	–	q : parcours complet (16 l.) ; r : index `ix_rdv_quai_creneau` (1 l.) ; p : index `fk_passage_quai` (100 l.) ; v : index `PRIMARY` (1 l.)
5	**Recherche plein texte (1re version)** — MATCH … AGAINST en sous-requête ; comparé à un LIKE '%…%'	0.40 ms	2.20 ms (×5.5)	article : index `ft_article_designation` (1 l.) ; ar : index `PRIMARY` (1 l.) ; c : index `PRIMARY` (1 l.) ; l : index `fk_lot_article` (1 l.)
6	**Recherche plein texte (version retenue)** — Étape 1 : articles par MATCH … AGAINST (index FULLTEXT)	0.06 ms	0.09 ms (×1.5)	article : index `ft_article_designation` (1 l.)

#	Requête	Temps médian	Sans index	Plan d'exécution
7	↳ Étape 2 : stock des articles trouvés (stock.php)	0.26 ms	–	ar : index `PRIMARY` (5 l.) ; c : index `PRIMARY` (1 l.) ; l : index `fk_lot_article` (1 l.) ; s : index `ix_stock_lot` (1 l.)
8	**Historique d'un lot** — Mouvements d'un lot par ordre chronologique (lot.php)	0.08 ms	1.74 ms (×20.6)	mouvement_stock : index `ix_mvt_lot_date` (22 l.)
9	**Mouvements des 7 derniers jours** — Historique filtré par date (mouvements.php)	1.05 ms	1.69 ms (×1.6)	mouvement_stock : index `ix_mvt_date` (2237 l.)
10	**Temps moyens par transporteur (30 j)** — Agrégation de v_passage_durees (indicateurs.php)	3.52 ms	–	v : index `fk_vehicule_transporteur` (123 l.) ; p : index `fk_passage_vehicule` (13 l.) ; arr : index `PRIMARY` (1 l.) ; appel : index `PRIMARY` (1 l.)
11	**Remplissage par zone** — Vue v_remplissage_zone (somme des palettes par zone)	5.38 ms	–	z : parcours complet (4 l.) ; : index `key0` (2 l.) ; : index `key0` (2 l.) ; z : index `PRIMARY` (1 l.)
12	**Taux d'écart par fournisseur** — Réceptions clôturées sur 30 jours (indicateurs.php)	2.47 ms	–	f : index `PRIMARY` (30 l.) ; a : index `fk_avis_fournisseur` (1 l.) ; r : index `fk_reception_avis` (1 l.) ; e : index `PRIMARY` (1 l.)
13	**Lots proches de la DLC** — Vue v_lots_proches_dlc (index sur la date limite)	1.54 ms	–	: parcours complet (341 l.) ; ar : parcours complet (341 l.) ; l : index `fk_lot_article|ix_lot_dlc` (1 l.) ; s : index `ix_stock_lot` (1 l.)
14	**Proposition d'emplacements** — Procédure stockée proposer_emplacements (rangement)	32.13 ms	–	procédure stockée
15	**Stock disponible d'un client** — Stock physique moins lignes réservées (saisie de commande)	1.17 ms	–	ar : index `fk_article_client` (35 l.) ; l : index `fk_lot_article` (1 l.) ; lp : index `fk_lpr_lot` (1 l.) ; li : index `PRIMARY` (1 l.)
16	**Contenu d'une expédition** — Unités chargées et poids par commande (documents de transport)	0.09 ms	–	ue : index `fk_ue_expedition` (5 l.) ; c : index `PRIMARY` (1 l.)

Analyse : recherche plein texte

La première version de la recherche de stock plaçait le `MATCH … AGAINST` dans une sous-requête `IN (…)`. Selon les données, l'optimiseur peut estimer une seule ligne pour la recherche plein texte, puis parcourir toute la chaîne zone → allée → rack → emplacement → stock avant de filtrer : c'est ce qui a été observé sur un jeu de données antérieur, où cette requête prenait 19 ms, soit plus qu'un simple `LIKE '%…%'` (5 ms). La version retenue résout d'abord les articles par l'index FULLTEXT, puis interroge le stock avec la liste d'identifiants obtenue : son plan ne dépend plus de cette estimation. Sur le jeu actuel, les lignes 5, 6 et 7 ci-

dessus montrent que les deux variantes sont équivalentes : le gain de la version retenue est la stabilité du plan, pas le temps moyen.

Analyse : requête la plus coûteuse

La proposition d'emplacements est la requête la plus lente : elle appelle `fn_palettes_libres` et `fn_poids_libre` pour chaque emplacement candidat, soit plus d'un millier d'appels. Elle reste de l'ordre de quelques dizaines de millisecondes et n'est exécutée qu'à l'ouverture de l'écran de rangement ; le coût est accepté pour garder ces règles dans la base.

Optimisations mises en place

- **Index composite** `ix_rdv_quai_creneau (quai_id, debut, fin)` : utilisé par le contrôle de chevauchement exécuté dans le déclencheur à chaque insertion ou modification de rendez-vous, et par le planning.
- **Index** `ix_mvt_lot_date (lot_id, date_heure)` : historique d'un lot déjà trié, sans tri supplémentaire (traçabilité).
- **Index** `ix_mvt_date`, `ix_pe_date`, `ix_rdv_debut` : filtres par période des écrans d'historique et des indicateurs.
- **Index FULLTEXT** `ft_article_designation` : recherche d'articles par mots, au lieu d'un `LIKE '%…%'` qui impose un parcours complet.
- **Index** `ix_lot_dlc` : détection des lots proches de leur date limite.
- **Dénormalisations contrôlées** : `passage.etape_courante_id` et `quai.statut` sont maintenus par déclencheur, ce qui évite de recalculer la dernière étape (sous-requête sur `passage_etape`) à chaque affichage des écrans temps réel ; la table `stock` est la synthèse des mouvements, maintenue par le déclencheur de `mouvement_stock`, pour ne pas sommer tout l'historique à chaque recherche.
- **Requêtes préparées** (PDO) : le plan est réutilisé et les paramètres ne sont jamais concaténés dans le SQL.